

✓ 1. Introduction to Object Oriented Programming.

You will be able to:

- Refactor existing code to use Object Oriented Programming principles.
- Add capabilities to the `datascience Table` class.
- Appreciate the benefits of OOP.
- Understand core OOP terminologies.

✓ 1.1 Review

We begin with a quick review of some of the concepts you should be familiar with.

1.1.1 Variables

In this module, we will use the term variable to refer to Python variables which are names that are given to values. The rules for naming a variable are:

- The first character must be a letter or an underscore.
- The following characters may be letters, numbers, or underscores.
- The name must not be a Python reserved word.

Note that spaces and special characters like `!`, `@`, `#`, `$`, `%`, `^`, `&`, `*`, `(`, and `)` are not permitted. By convention, use lower case letters and separate words with underscores e.g `age_limit`. The official styling guide for writing Python code is [Python Enhancement Proposal 8 \(PEP 8\)](#).

[More on variables](#)

✓ 1.1.2 Lists

Consider a dataset containing the ages of 10 students. If we are to assign each age a variable, we'll need 10 different variables. The disadvantages of doing this are obvious. A list provides a way to name a structure that can reference the 10 ages.

To create a list, we use brackets with the values (elements) inside separated by commas.

```
# List of the ten ages.  
ages = [26, 25, 27, 25, 20, 22, 29, 30, 22, 29]
```

To access an element in the list, we use its name and index, that is `list_name[index]`. The indexes start from 0 for first value, 1 for second value, 2 for third value, and so on.

```
# Display ages of the first and fourth elements of the list using the print function
print(age[0])
print(age[4])
```

Output:

```
26
20
```

In the above example, we used the print function to display the values on the screen. To compute summary values, we use other functions like `len()`, `min()`, `max()`, `sum()`, and so on. As an example, to display the minimum age on our list, we write

```
print(min(age))
```

and get

```
20
```

Exercise 1:

1. Run the code below to view the reserved words in python.
2. Modify the code to display how many they are.
3. (Optional) Display the reserved words in long format, i.e, one per line.
4. Test.

```
"""Print a list of Python reserved words """
```

```
import keyword          # Makes keyword code, written by others, available to this program.
```

```
python_keywords = keyword.kwlist
print(python_keywords)
```

```
# TODO: Add a line of code below to print the number of Python keywords.
```

```
# TODO (Optional): Add lines of code to display the reserved words in long format, i.e, one
```

[More on data structures](#)

✓ 1.1.3 Functions

We have called functions like `min()` and `print()`. To write our own functions, we use the `def` keyword followed by the name of the function, followed by the parameters in parenthesis. Parameters are separated by commas. Here is the syntax for creating a function.

```
def function_name(param_1, param_2, ..., param_n):  
    # body of function goes here
```

Note:

- The rules for naming a function is that same as for naming variables.
- `return` keyword may be used within the body to stop execution and return a value.
- Parameters can have initial values.

[More on functions](#)

Example:

To compute the variance and standard deviation of the ages of a sample of students, we write two functions. One that computes the variance and the other that computes the standard deviation. The ages can be stored as a list and given an appropriate name. Finally, we call the two functions, passing them the list and displaying what they return.

The formulae are:

- $\text{Variance} = \frac{\sum (X - \bar{X})^2}{n - 1}$, where n is the sample size and $\bar{X}$ is the mean.
- The standard deviation is the square root of the variance.

```
# Define the functions.
```

```
def variance(data):  
    '''returns the variance of the data'''  
    mean = sum(data) / len(data)  
    return sum((x - mean) ** 2 for x in data) / (len(data) - 1)  
  
def standard_deviation(data):  
    '''returns the standard deviation using the variance function'''  
    return variance(data) ** 0.5
```

```
# Use a list to store the dataset.
```

```
ages = [26, 25, 27, 25, 20, 22, 29, 30, 22, 29]
```

```
# Call above functions passing them the list and printing what they return.  
print('Variance:', variance(ages))  
print('Standard Deviation:', standard_deviation(ages))
```

```
➡ Variance: 11.388888888888889  
Standard Deviation: 3.3747427885527643
```

Functions provide several advantages including:

1. **Reusability.** As an example, to compute the standard score of a given value,

$$\text{standard score} = \frac{\text{value} - \text{mean}}{\text{standard deviation}}$$

we would reuse the variance and standard deviation functions. In fact, we could **decompose** the variance function to create a new function that just returns the mean and re-use it too.

2. **Modularity.** Each function is responsible for a specific task which results in modular code.
3. **Easier debugging** as a bug can be easily traced to a particular function.
4. **Easier testing** as a function can be tested independently.
5. **Collaboration.** Different people can work on different functions that belong to the same project.

Exercise 2:

Write a function, with two parameters data and age, which computes and returns the standard score of a given age. Test it. Write another function that returns a list of standard scores from the given data list.

```
# Define a function with two parameters that returns the standard score if given one of the
```

✓ 1.2 Classes and Objects

Next, we discuss classes and objects which form the basis of Object-Oriented Programming.

1.2.1 Class

We now bundle the functions and variables into a structure called a class and give it a name. By doing so, we reap the benefits that functions come along with. We call these functions **methods** and the variables **attributes**.

To declare a class, we use the `class` keyword followed by a space and the name as follows.

```
class ClassName:
    # class body goes here
```

Note:

- The rules for naming a class is the same as the rules for naming variables.
- Class names use CapWords by [convention](#) e.g. ShoppingCart.
- It is good practice to add some documentation to a class.

Since `ages` list we used earlier normally corresponds to a column in a dataset, we will name our first class `Column`. We declare the class as follows:

```
class Column:
    """An empty class"""
    pass
```

Exercise 3:

Identify which of the following names are valid class names. Which ones follow the convention rules?

- A. table
- B. _table
- C. __
- D. Data_Set
- E. DataSet
- F. Table
- G. Class
- H. class

Double-click (or enter) to edit

✓ 1.2.2 Objects

After defining a class, we can then create an object by declaring a variable to reference the object and assigning it the name of the class with parenthesis as follows:

```
object_name = ClassName()
```

Example:

The code below creates two objects of `Column` class

```
# First object is ages which is an instance of Column class.
ages = Column()

# Second object is heights, another instance of Column class.
heights = Column()
```

We say that `ages` and `heights` are **instances** of `Column` class.

Example

1. Run the code below to create two objects and display the data type of one of the objects.
2. Add code to display the data type of height.
3. What can you conclude from the output?

```
class Column:
    """An empty class"""
    pass

# Create the two objects
ages = Column()
heights = Column()

# Display data type of ages.
print(type(ages))

# TODO: Display the data type of heights.
```

It turns out that classes introduce new data types in addition to **int**, **float**, **str**, and **bool**.

Let us now modify the `Column` class by adding a few methods in the body. Since we would like to give each instance of `Column` a title at the time they are created, we must name our method `__init__`. Yes, it has two underscores on either side. This special method is called a **constructor**. It is automatically executed once whenever a new object is created.

The first parameter of a method is a reference to the current object as there can be many objects created from a class. By convention, it is named `self`. Other parameters follow it if needed.

Inside the constructor, we initialize the attributes. To access or create an attribute, we use the dot notation i.e. `object_name.instance_variable`. Inside a class method, we use `self.instance_variable` as `self` references the current object.

```
class Column:
    def __init__(self, title):      # title must be supplied at creation.
        self.label = title

# When calling a method, self argument is automatically passed.
ages = Column('Age')
```

Note:

- **self.label** is the attribute and can be accessed by other methods within the `Column` class.
- Each object created will have its own **label**.
- All variables with `self.` prefixes are instance variables.
- Special methods in Python begin with double underscores.
- When calling a method, supply one less argument as the first is automatically passed.

Example:

1. Run the following code to display the title of the age column.
2. Add a line to create `heights` object with the title `Height`.
3. Add a line to display the title of the `heights` column.
4. Rerun to verify.
5. Modify the constructor to accept an additional parameter. Assign its value to the instance variable `data`.

```
class Column:
    """ Represents a column of a dataset """

    def __init__(self, title):
        '''This is the constructor'''
        self.label = title

# Create two objects with appropriate titles required .
ages = Column('Age')

# TODO: add a line to create heights object with the title Height.

# Display the title of age column
print(ages.label)

# TODO: Add the line to display the title of the height column.
```

```
class Column:
    def __init__(self, title, data):
        self.label = title
        self.data = data
```

It is perfectly acceptable to use **self.data** as the attribute name and **data** as the method parameter. They reference two different things.

Let us now add the variance and standard deviation methods. Take note of how the `standard_deviation` method calls the `variance` method.

```
class Column:
    """ Represents a column of a dataset """
    def __init__(self, title, data=[]):
        self.label = title
        self.data = data          # data and self.data are different.

    def variance(self):
        '''returns the variance'''
        mean = sum(self.data) / len(self.data)
        return sum((x - mean) ** 2 for x in self.data) / len(self.data)

    def standard_deviation(self):
        '''returns the standard deviation'''
        return self.variance() ** 0.5    # variance method called here.
```

Note:

- The variable `mean` inside `variance` method is local to the function. Its scope is within the function.

Exercise 4:

One of the ideas behind OOP paradigm is to mimic real life objects interacting with each other to accomplish tasks. As an example, you as an object get in your car (another object) in the morning, drive to school yet another object where you interact with other objects to accomplish your work for the day.

The car has its own **attributes** e.g. make, model, color, etc. You have your own attributes. When it comes to **behavior** the car can accelerate, stop, turn, etc. you can also do the same in addition to talking.

Think of two objects you might interact with at school. List them together with their attributes and behaviors.

Double click here to type your response.

Exercise 5: Modify `Column` class as follows:

- Add a new method, named `set_column_type`, that sets an attribute, named `column_type`, by looking at the first element of the list.
- Call the method from the constructor.
- Test.

```
class Column:
    """ Represents a column of a dataset"""
    def __init__(self, title, data):
        self.label = title
        self.data = data
        # TODO 1.: Call the method to set the data type of the column.

    #TODO 2.: Create the function, set_column_type.

    def variance(self):
        '''returns the variance'''
        mean = sum(self.data) / len(self.data)
        return sum((x - mean) ** 2 for x in self.data) / len(self.data)

    def standard_deviation(self):
        '''returns the standard deviation'''
        return self.variance() ** 0.5

# TODO: Test Column type
ages = Column('Age', [26, 25, 27, 25, 20, 22, 29, 30, 22, 29])
genders = Column('Gender', ['M', 'F', 'F', 'M', 'M', 'F', 'M', 'M', 'M', 'M'])

print(ages.column_type)
print(genders.column_type)
```

Exercise 6:

Modify `Column` class so that:

- All the data in the list have the same data type.
- Division by zero in computations are taken care of.

- Computations can only be done on columns with numeric values.

Start coding or [generate](#) with AI.

Exercise 7:

Add a special method called `__str__` that must return a string which is the label and its data values in long format in this case. Whenever an object is printed, this method is executed automatically, and its string value is displayed. Test it as follows:

```
ages = Column('Age', [26, 25, 27, 25, 20, 22, 29, 30, 22, 29])
print(ages)
```

Output

```
Age
26
25
27
25
20
22
30
22
29
29
```

Start coding or [generate](#) with AI.

Exercise 8:

Write five methods inside `Column` that computes and returns the Five-Number summary i.e. smallest, first quartile (Q1), median, third quartile (Q3) and the largest value.

Exercise 9:

Write a method inside `Column` to detect outliers and test.

Exercise 10:

(Optional) Add other methods that you think may be useful.

Exercise 11:

Write `Column` class in its own file e.g. `column.py`. Open another file and import it as follows. Test it by creating a `Column` object. You may need to modify your path environment.

```
from column import Column
```

```
# code goes here
```

✓ 1.2.3 Class Variables

Suppose we would like to keep count of the total number of `Column` instances created. We need a variable that is visible to all objects. This is achieved by declaring it outside of all the methods. Each time a new object is created we increment the counter by 1.

```
class Column:
    num_columns = 0                # class variable

    def __init__(self, title, data):
        Column.num_columns += 1    # Increment by 1
```

Note:

- `num_columns` is initialized to 0 once when the first object is created.
- `num_columns` is called a class variable.
- We use the class name `Columns` and not `self` to refer to a class variable.

Example:

Run the code. Create a new column and re-run the code. Take note of the number of columns displayed.

```
class Column:
    num_columns = 0

    def __init__(self, title, data):
        Column.num_columns += 1

# Create three Column objects
ages = Column('Age', [26, 25, 27, 25, 20, 22, 29, 30, 22, 29])
heights = Column('Height', [68.2, 64.7, 66.1, 70.3, 65.4, 63.9, 66.9, 67.2, 69.1, 64.3])
genders = Column('Gender', ['M', 'F', 'F', 'M', 'M', 'F', 'M', 'M', 'M', 'M'])

# TODO 1: Add a new Column object
```

```
print('Number of Columns:', Column.num_columns)
```

✓ 1.2.4 Member Access

Class members prefixed by `__` (double underscore) are **private** and can only be accessed within the class. Those that are prefixed by `_` (single underscore) are **protected** and can only be accessed within the class as well as its child classes.

```
class Column:
    def __init__(self, title, data):
        self._label = title          # Protected
        self.data = data

class Dataset:
    """ Represents the dataset as a list of Column objects """
    def __init__(self, data=None):
        self.__columns = data        # Private variable
```

Example

Run the following code then change `self.data` to `self.__data` on the indicated lines and re-run. Read the error message.

```
class Column:
    def __init__(self, title, data):
        self.label = title           # TODO 1: change self.label to self.__label
        self.data = data

    def __str__(self):
        return self.label + ' : ' + str(self.data)

class Dataset:
    """ Represents a dataset as a list of Column objects """
    def __init__(self, data=None):
        self.columns = data

ages = Column('Age', [1, 2, 3, 4, 6, 7, 8, 10])
genders = Column('Gender', ['Male', 'Female', 'Female', 'Male', 'Male', 'Female'])

table = Dataset([ages, genders])
```

```
# Display the first columns label
print(table.columns[0].label)      # TODO 2: change table.columns[0].label to table.columns[0].label
```

Exercise 12:

Add a special method `__repr_html__()` that returns the HTML representation of a dataset in tabular form. To test it, the following should display the column.

```
ages = Column('Age', [1, 2, 3, 4, 6, 7, 8, 10])
age
```

Exercise 13:

Add a method to add grouping functionality. The method should accept a column label and return a `Dataset` where the first column contains the unique values from the column, and the second contains counts for each of the unique values.

Exercise 14:

Modify the `Dataset` so that it can read a dictionary dataset where the key is label and the value is a list. The label in `Column` can be used for the units.

Exercise 15:

Modify `Dataset` so that it can export data to pandas dataframe. (Optional) Export to other formats like excel, csv, etc.

✓ 1.2.5 Inheritance

The `datascience` module has a `Table` class that is more polished than our `Dataset` class. Instead of reinventing the wheel, we will use the `Table` class from now on. `Table` class is not perfect though. As an example, it cannot be used to create a table from an excel file. Let us introduce a new class, `XTable`, that will provide additional capabilities.

```
# Import the Table class
from datascience import Table

# Define the new table that extends Table
class XTable(Table):
    pass
```

Note:

- `Table` is in parenthesis of `XTable`

Run the following code to demonstrate that `XTable` and `Table` have the same capabilities. Modify it to convince yourself.

```
from datascience import Table

class XTable(Table):
    """ This class extends Table class in the datascience module"""
    pass

data = XTable().with_columns('Age', [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                             'Gender', ['M', 'F', 'F', 'M', 'M', 'F', 'M', 'M', 'M', 'M'])
data.group('Gender')
```

Note:

- `Table` is called the parent (or base) class while `XTable` is the child (or subclass).
- `XTable` inherits all the members of `Table`.

To read datasets from excel files, we add a new method `from_excel()` which uses the `read_excel()` method from `pandas` package. [Python Data Analysis Library \(pandas\)](#) is a free python library used for data analysis and manipulation.

```
from datascience import Table
import pandas as pd

class XTable(Table):
    """ This class extends Table class in the datascience module"""

    def from_excel(self, file):
        ''' creates a table from an excel file using pandas. '''
        return Table.read_table(pd.read_excel(file))
```

Let us add another capability. `Table` has a method called `join` that only supports inner joins and its signature is:

```
Table.join(column_label, other, other_label=None)
```

It basically creates a new table with the columns of `self` and `other`, containing rows for all values of a column with `column_label` that appear in both tables.

We would like to support **left**, **right**, and **outer** joins as well. We therefore create a new method but with the same name this time. When a `XTable` object is created and it calls the `join` method, which

join method is called? The one in `XTable` (the child class). We say the `join` method in `XTable` **overrides** the one in `Table` (parent). If we still need to call the `join()` in the parent class, we use `super().join()`

```
# import Table and pandas
from datascience import Table
import pandas as pd

# Define the new table that extends Table
class XTable(Table):
    """ This class extends Table class in the datascience module"""

    def from_excel(self, file):
        ''' creates a table from an excel file using pandas'''
        return Table.read_table(pd.read_excel(file))

    def join(self, column_label, other, other_column_label=None, how='inner'):
        ''' Uses pandas join to combine two tables '''
        if how=='inner':

            # Call join in Table class using supper()
            return super().join(column_label, other, other_column_label)
        else:
            dataframe = self.to_df().merge(other.to_df(),on=column_label, how=how)
            return self.from_df(dataframe)

# Sample dataset
climate = XTable().with_columns( 'Region', ['North', 'South', 'East', 'Northeast', 'West'],
                                'Temperature', [12.3, 22.1, 18.5, 15.4, 19.0])

regions = XTable().with_columns('Region', ['North', 'South', 'East', 'West', 'Central'],
                                'Population', [120000, 95000, 134000, 87000, 110000])

# TODO: Change the value of the agrument to 'left', 'right', 'inner', 'outer' in turn and ru
climate_regions = climate.join("Region", regions, how='inner')
climate_regions.show()
```



Region	Temperature	Population
East	18.5	134000
North	12.3	120000
South	22.1	95000
West	19	87000

Sometimes, in a method, we may need to execute the parent class method first. As an example, if we create a constructor in the `XTable`, we most likely need to execute the parents' constructor first

as follows.

```
class XTable(Table):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

    # Other code follows.
```

If the child class has an attribute with the same name as an attribute in the parent class, then we say the child class's attribute **shadows** the parent's one. To access the parent one use `super()`.

Exercise 16:

What is the advantage of introducing a child (`XTable`) instead of modifying the parent class (`Table`).

Exercise 17:

Given the `TableCleaner` class below, create a subclass called `SurveyCleaner` that inherits from `TableCleaner`.

- In `SurveyCleaner`, add a method `encode_gender()` that adds a new column called `Gender_Code` with values 1 for 'Male', 0 for 'Female' and -1 for other or blank values.
- Override the `drop_rows_with_empty_values()` method so that it only drops rows where `Age` or `Satisfaction_Score` is empty, ignoring missing values in other columns.
- Test your code with the following sample dataset

```
from datascience import Table
```

```
class TableCleaner:
    def __init__(self, filepath):
        self.table = Table.read_table(filepath)

    def drop_rows_with_empty_values(self, column_labels):
        """Drops rows that have empty values ('') or None in the specified columns."""
        for label in column_labels:
            non_empty = []
            for value in self.table.column(label):
                non_empty.append(value != '' and value is not None)
            self.table = self.table.where(non_empty)

    def get_table(self):
        return self.table
```



```
Name, Age, Gender, Country, Satisfaction_Score
Alice, 29, Female, USA, 8
Bob, 35, Male, , 7
Charlie, , Non-binary, Canada,
David, 42, Male, UK, 9
Eve, 30, Female, , 6
Frank, 27, , Mexico, 5
```

Exercise 18:

Modify `xTable` so that it can read Stata datasets. (Optional) Add capabilities of reading datasets in `json`, and `XML` formats, from REST APIs and a relational database system.